

IMPORT LIBRARIES

```
In [90]: import warnings
warnings.filterwarnings("ignore")

# Core
import numpy as np
import pandas as pd

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Sklearn
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, VotingClassifier

from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    roc_auc_score,
    roc_curve,
    precision_recall_curve,
    auc,
    f1_score,
    precision_score,
    recall_score
)

# Imbalanced Learning
from imblearn.over_sampling import SMOTE

# XGBoost
from xgboost import XGBClassifier
```

```
# Deep Learning
import tensorflow as tf
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Dense, Dropout, Input
from tensorflow.keras.callbacks import EarlyStopping

# Save Models
import joblib
```

Load Dataset

```
In [91]: df=pd.read_csv(r'C:\Users\anupa\Desktop\Python Learning\Fraud\creditcard.csv', low_memory=False)
print("="*60)
print("DATASET SHAPE")
print("="*60)
print(df.shape)
```

```
=====
DATASET SHAPE
=====
```

```
(284807, 31)
```

BASIC DATA ANALYSIS

```
In [92]: print("\nFIRST 5 ROWS\n")
print(df.head())

print("\nINFO\n")
print(df.info())

print("\nDESCRIPTION\n")
print(df.describe())

print("\nMISSING VALUES\n")
print(df.isnull().sum())
```

```
print("\nCLASS DISTRIBUTION\n")  
print(df["Class"].value_counts())
```

FIRST 5 ROWS

	Time	V1	V2	V3	V4	V5	V6	V7	\
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	

	V8	V9	...	V21	V22	V23	V24	V25	\
0	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	
1	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	
2	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	
3	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	
4	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	

	V26	V27	V28	Amount	Class
0	-0.189115	0.133558	-0.021053	149.62	0
1	0.125895	-0.008983	0.014724	2.69	0
2	-0.139097	-0.055353	-0.059752	378.66	0
3	-0.221929	0.062723	0.061458	123.50	0
4	0.502292	0.219422	0.215153	69.99	0

[5 rows x 31 columns]

INFO

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
#   Column  Non-Null Count  Dtype
---  -
0    Time    284807 non-null  float64
1    V1       284807 non-null  float64
2    V2       284807 non-null  float64
3    V3       284807 non-null  float64
4    V4       284807 non-null  float64
5    V5       284807 non-null  float64
6    V6       284807 non-null  float64
7    V7       284807 non-null  float64
8    V8       284807 non-null  float64
```

```

9   V9      284807 non-null float64
10  V10     284807 non-null float64
11  V11     284807 non-null float64
12  V12     284807 non-null float64
13  V13     284807 non-null float64
14  V14     284807 non-null float64
15  V15     284807 non-null float64
16  V16     284807 non-null float64
17  V17     284807 non-null float64
18  V18     284807 non-null float64
19  V19     284807 non-null float64
20  V20     284807 non-null float64
21  V21     284807 non-null float64
22  V22     284807 non-null float64
23  V23     284807 non-null float64
24  V24     284807 non-null float64
25  V25     284807 non-null float64
26  V26     284807 non-null float64
27  V27     284807 non-null float64
28  V28     284807 non-null float64
29  Amount  284807 non-null float64
30  Class   284807 non-null int64

```

dtypes: float64(30), int64(1)

memory usage: 67.4 MB

None

DESCRIPTION

	Time	V1	V2	V3	V4 \
count	284807.000000	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05
mean	94813.859575	1.175161e-15	3.384974e-16	-1.379537e-15	2.094852e-15
std	47488.145955	1.958696e+00	1.651309e+00	1.516255e+00	1.415869e+00
min	0.000000	-5.640751e+01	-7.271573e+01	-4.832559e+01	-5.683171e+00
25%	54201.500000	-9.203734e-01	-5.985499e-01	-8.903648e-01	-8.486401e-01
50%	84692.000000	1.810880e-02	6.548556e-02	1.798463e-01	-1.984653e-02
75%	139320.500000	1.315642e+00	8.037239e-01	1.027196e+00	7.433413e-01
max	172792.000000	2.454930e+00	2.205773e+01	9.382558e+00	1.687534e+01

	V5	V6	V7	V8	V9 \
count	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05
mean	1.021879e-15	1.494498e-15	-5.620335e-16	1.149614e-16	-2.414189e-15

std	1.380247e+00	1.332271e+00	1.237094e+00	1.194353e+00	1.098632e+00
min	-1.137433e+02	-2.616051e+01	-4.355724e+01	-7.321672e+01	-1.343407e+01
25%	-6.915971e-01	-7.682956e-01	-5.540759e-01	-2.086297e-01	-6.430976e-01
50%	-5.433583e-02	-2.741871e-01	4.010308e-02	2.235804e-02	-5.142873e-02
75%	6.119264e-01	3.985649e-01	5.704361e-01	3.273459e-01	5.971390e-01
max	3.480167e+01	7.330163e+01	1.205895e+02	2.000721e+01	1.559499e+01

	...	V21	V22	V23	V24 \
count	...	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05
mean	...	1.628620e-16	-3.576577e-16	2.618565e-16	4.473914e-15
std	...	7.345240e-01	7.257016e-01	6.244603e-01	6.056471e-01
min	...	-3.483038e+01	-1.093314e+01	-4.480774e+01	-2.836627e+00
25%	...	-2.283949e-01	-5.423504e-01	-1.618463e-01	-3.545861e-01
50%	...	-2.945017e-02	6.781943e-03	-1.119293e-02	4.097606e-02
75%	...	1.863772e-01	5.285536e-01	1.476421e-01	4.395266e-01
max	...	2.720284e+01	1.050309e+01	2.252841e+01	4.584549e+00

		V25	V26	V27	V28	Amount \
count	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	284807.000000	
mean	5.109395e-16	1.686100e-15	-3.661401e-16	-1.227452e-16	88.349619	
std	5.212781e-01	4.822270e-01	4.036325e-01	3.300833e-01	250.120109	
min	-1.029540e+01	-2.604551e+00	-2.256568e+01	-1.543008e+01	0.000000	
25%	-3.171451e-01	-3.269839e-01	-7.083953e-02	-5.295979e-02	5.600000	
50%	1.659350e-02	-5.213911e-02	1.342146e-03	1.124383e-02	22.000000	
75%	3.507156e-01	2.409522e-01	9.104512e-02	7.827995e-02	77.165000	
max	7.519589e+00	3.517346e+00	3.161220e+01	3.384781e+01	25691.160000	

	Class
count	284807.000000
mean	0.001727
std	0.041527
min	0.000000
25%	0.000000
50%	0.000000
75%	0.000000
max	1.000000

[8 rows x 31 columns]

MISSING VALUES

Time	0
V1	0
V2	0
V3	0
V4	0
V5	0
V6	0
V7	0
V8	0
V9	0
V10	0
V11	0
V12	0
V13	0
V14	0
V15	0
V16	0
V17	0
V18	0
V19	0
V20	0
V21	0
V22	0
V23	0
V24	0
V25	0
V26	0
V27	0
V28	0
Amount	0
Class	0

dtype: int64

CLASS DISTRIBUTION

Class	
0	284315
1	492

Name: count, dtype: int64

Visualize imbalance

```
In [93]: plt.figure(figsize=(6,4))  
  
sns.countplot(x="Class", data=df)  
  
plt.title("Fraud vs Genuine Transactions")  
  
plt.show()
```



Split Features and Target

```
In [94]: X = df.drop("Class", axis=1)  
y = df["Class"]
```


Train-test split

```
In [95]: X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.2,
        stratify=y,
        random_state=42
    )

print("\nTRAIN SHAPE:", X_train.shape)
print("TEST SHAPE:", X_test.shape)
```

TRAIN SHAPE: (227845, 30)

TEST SHAPE: (56962, 30)

FEATURE SCALING

```
In [96]: scaler = StandardScaler()

# Scale Amount and Time AFTER split
X_train[['Amount', 'Time']] = scaler.fit_transform(
    X_train[['Amount', 'Time']]
)

X_test[['Amount', 'Time']] = scaler.transform(
    X_test[['Amount', 'Time']]
)
```

Handle Imbalanced Data Using SMOTE

```
In [97]: #Fraud datasets are highly imbalanced.
#SMOTE generates synthetic minority class examples.
smote = SMOTE(random_state=42)

X_train_smote, y_train_smote = smote.fit_resample(
    X_train,
```

```
y_train
)

print("\nBEFORE SMOTE")
print(y_train.value_counts())

print("\nAFTER SMOTE")
print(y_train_smote.value_counts())
```

BEFORE SMOTE

Class

0 227451

1 394

Name: count, dtype: int64

AFTER SMOTE

Class

0 227451

1 227451

Name: count, dtype: int64

Train Logistic Regression Model

```
In [98]: print("\n" + "="*60)
print("LOGISTIC REGRESSION")
print("="*60)
```

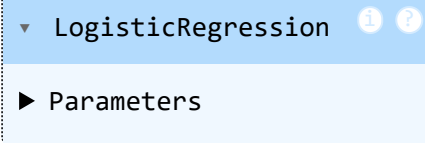
```
# -----
# Train Model
# -----
```

```
lr_model = LogisticRegression(
    max_iter=1000,
    random_state=42
)
```

```
lr_model.fit(
    X_train_smote,
    y_train_smote
)
```

```
=====
LOGISTIC REGRESSION
=====
```

Out[98]:



Predict Probabilities

```
In [99]: y_prob_lr = lr_model.predict_proba(X_test)[:, 1]
```

Precision Recall Curve

```
In [100... precision_lr, recall_lr, thresholds_lr = precision_recall_curve(
    y_test,
    y_prob_lr
)
```

F1 Score for Every Threshold

```
In [101... f1_scores_lr = (
    2 * precision_lr[:-1] * recall_lr[:-1]
) / (
    precision_lr[:-1] + recall_lr[:-1] + 1e-10
)
```

Best Threshold

```
In [102... best_idx_lr = np.argmax(f1_scores_lr)

best_threshold_lr = thresholds_lr[best_idx_lr]

print("\nBest Threshold:", best_threshold_lr)
```

Best Threshold: 0.9999999906914586

Final Predictions

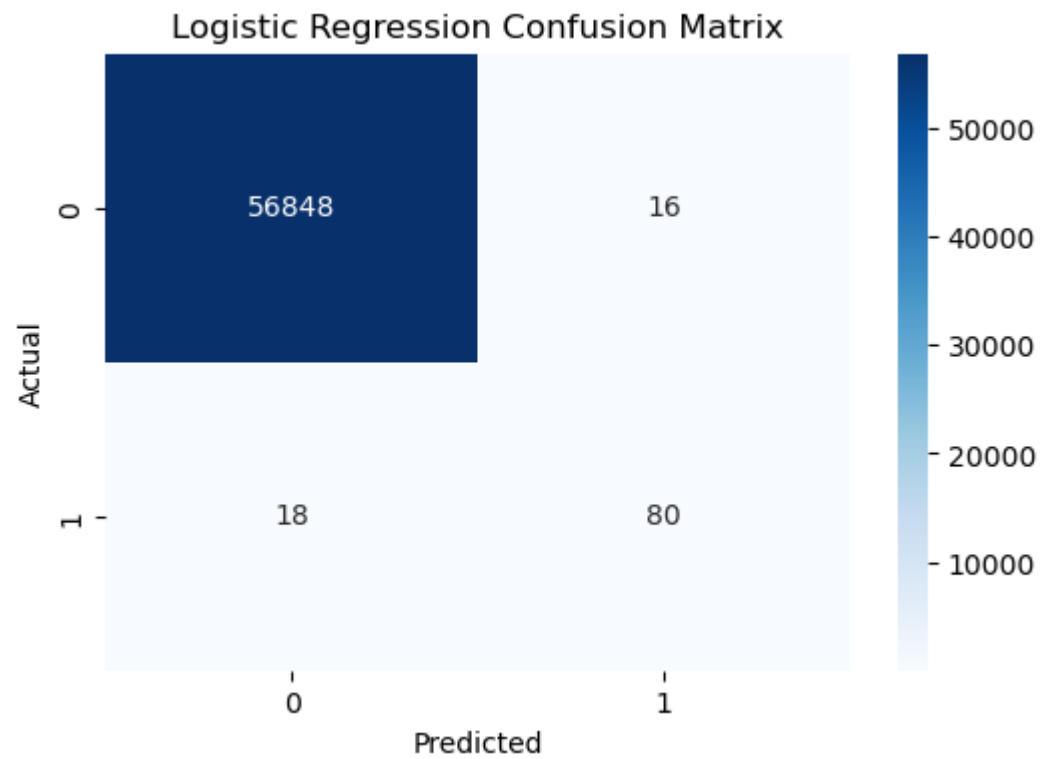
```
In [103... y_pred_lr = (  
    y_prob_lr >= best_threshold_lr  
) .astype(int)
```

Confusion Matrix

```
In [104... print("\nCONFUSION MATRIX\n")  
  
cm_lr = confusion_matrix(y_test, y_pred_lr)  
  
print(cm_lr)  
  
plt.figure(figsize=(6,4))  
  
sns.heatmap(  
    cm_lr,  
    annot=True,  
    fmt='d',  
    cmap='Blues'  
)  
  
plt.title("Logistic Regression Confusion Matrix")  
  
plt.xlabel("Predicted")  
plt.ylabel("Actual")  
  
plt.show()
```

CONFUSION MATRIX

```
[[56848  16]  
 [  18  80]]
```



Classification Report

```
In [105... print("\nCLASSIFICATION REPORT\n")

print(classification_report(y_test, y_pred_lr))

print(
    "Precision:",
    precision_score(y_test, y_pred_lr)
)

print(
    "Recall:",
    recall_score(y_test, y_pred_lr)
)
```

```

print(
    "F1 Score:",
    f1_score(y_test, y_pred_lr)
)

print(
    "ROC-AUC:",
    roc_auc_score(y_test, y_prob_lr)
)

```

CLASSIFICATION REPORT

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.83	0.82	0.82	98
accuracy			1.00	56962
macro avg	0.92	0.91	0.91	56962
weighted avg	1.00	1.00	1.00	56962

Precision: 0.8333333333333334

Recall: 0.8163265306122449

F1 Score: 0.8247422680412371

ROC-AUC: 0.9698631105509169

ROC CURVE

In [106...

```

# =====
# ROC CURVE WITH BEST THRESHOLD
# =====

# ROC values
fpr_lr, tpr_lr, roc_thresholds_lr = roc_curve(
    y_test,
    y_prob_lr
)

# ROC-AUC Score

```

```
roc_auc_lr = roc_auc_score(
    y_test,
    y_prob_lr
)

# -----
# Find ROC Point Closest to Best Threshold
# -----

roc_best_idx = np.argmin(
    np.abs(roc_thresholds_lr - best_threshold_lr)
)

best_fpr = fpr_lr[roc_best_idx]
best_tpr = tpr_lr[roc_best_idx]

# -----
# Plot ROC Curve
# -----

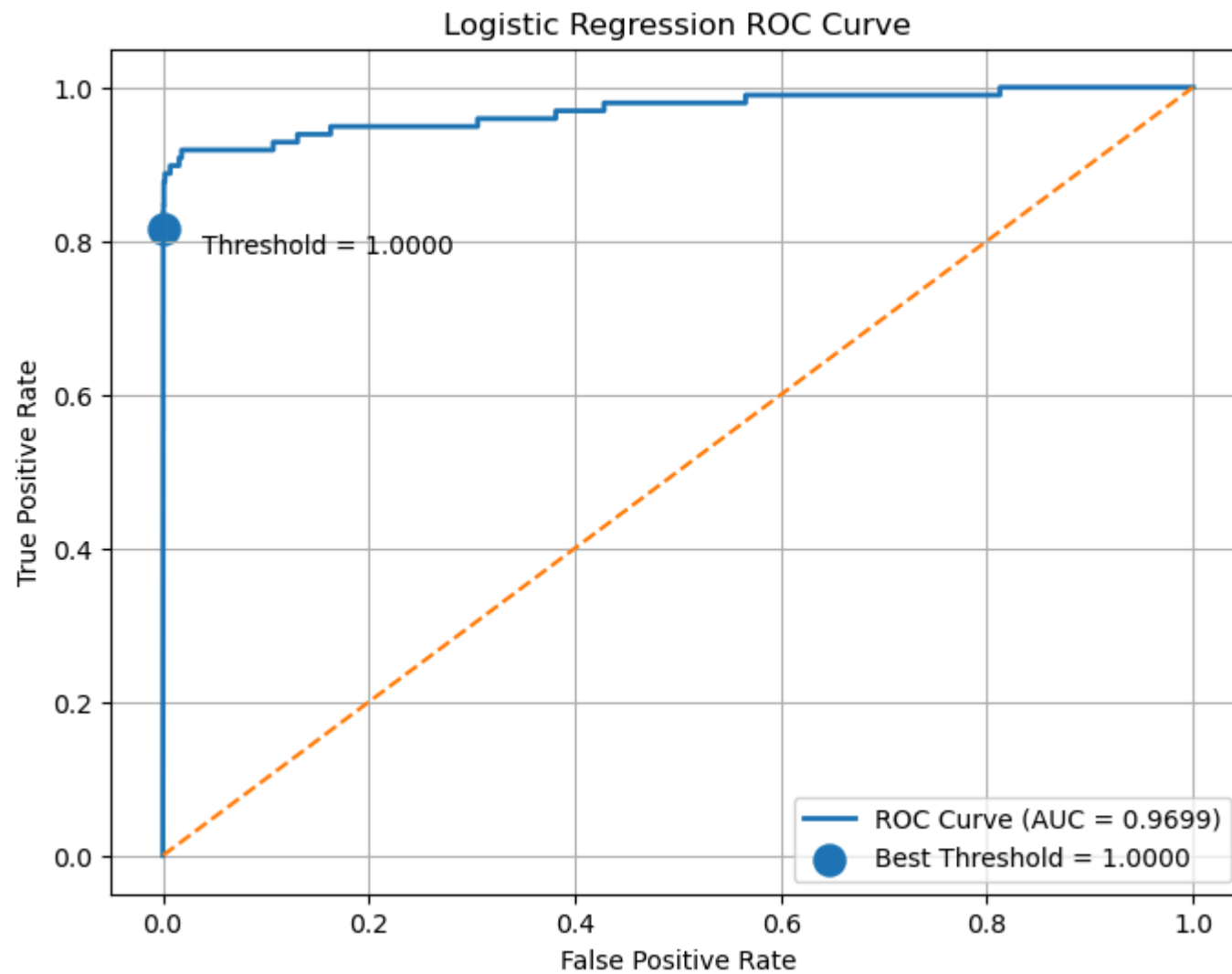
plt.figure(figsize=(8,6))

# ROC Curve
plt.plot(
    fpr_lr,
    tpr_lr,
    linewidth=2,
    label=f"ROC Curve (AUC = {roc_auc_lr:.4f})"
)

# Random Guess Line
plt.plot(
    [0,1],
    [0,1],
    linestyle='--'
)

# -----
# Best Threshold Point
# -----
```

```
plt.scatter(  
    best_fpr,  
    best_tpr,  
    s=150,  
    marker='o',  
    label=f"Best Threshold = {best_threshold_lr:.4f}"  
)  
  
# -----  
# Annotate Threshold  
# -----  
  
plt.annotate(  
    f"Threshold = {best_threshold_lr:.4f}",  
    (best_fpr, best_tpr),  
    textcoords="offset points",  
    xytext=(15,-10),  
    fontsize=10  
)  
  
# -----  
# Labels  
# -----  
  
plt.xlabel("False Positive Rate")  
  
plt.ylabel("True Positive Rate")  
  
plt.title("Logistic Regression ROC Curve")  
  
plt.legend()  
  
plt.grid(True)  
  
plt.show()
```

Precision-Recall Curve

```
In [107... pr_auc_lr = auc(  
    recall_lr,  
    precision_lr  
)
```

```
plt.figure(figsize=(8,6))

plt.plot(
    recall_lr,
    precision_lr,
    label=f"PR-AUC = {pr_auc_lr:.4f}"
)

plt.scatter(
    recall_lr[best_idx_lr],
    precision_lr[best_idx_lr],
    s=120,
    label=f"Threshold={best_threshold_lr:.4f}"
)

plt.xlabel("Recall")

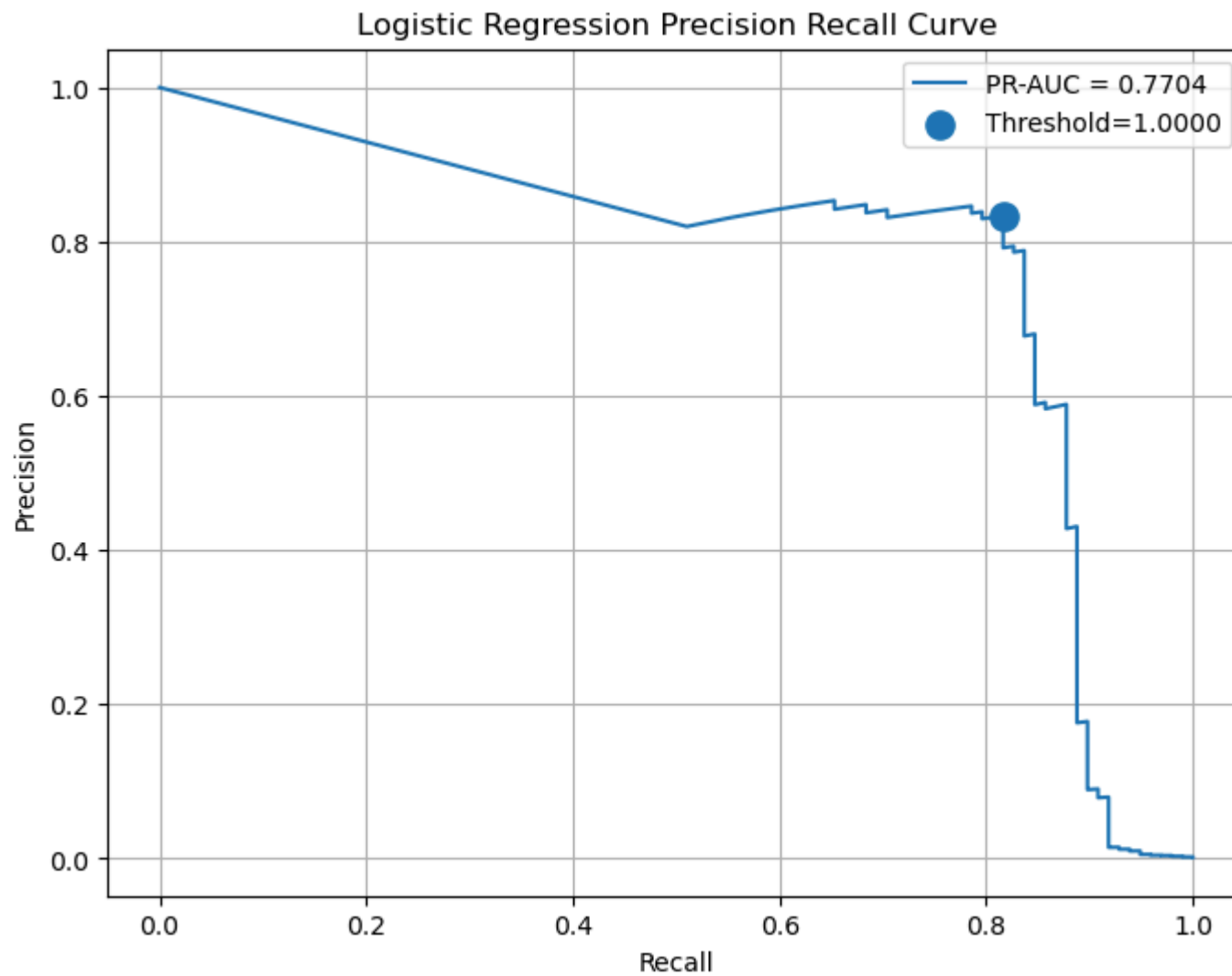
plt.ylabel("Precision")

plt.title("Logistic Regression Precision Recall Curve")

plt.legend()

plt.grid(True)

plt.show()
```



Train Random Forest Model

```
In [108... print("\n" + "="*60)
print("RANDOM FOREST")
print("="*60)
```

```
# -----  
# Train Model  
# -----  
  
rf_model = RandomForestClassifier(  
    n_estimators=200,  
    random_state=42,  
    class_weight='balanced',  
    n_jobs=-1  
)  
  
rf_model.fit(  
    X_train_smote,  
    y_train_smote  
)
```

```
=====
```

RANDOM FOREST

```
=====
```

Out[108...

RandomForestClassifier ⓘ ?

► Parameters

Predict Probabilities

In [109...

```
y_pred_rf = rf_model.predict(X_test)  
y_prob_rf = rf_model.predict_proba(X_test)[:, 1]
```

Precision Recall Curve

In [110...

```
precision_rf, recall_rf, thresholds_rf = precision_recall_curve(  
    y_test,  
    y_prob_rf  
)
```

F1 Scores

```
In [111... f1_scores_rf = (  
    2 * precision_rf[:-1] * recall_rf[:-1]  
) / (  
    precision_rf[:-1] + recall_rf[:-1] + 1e-10  
)
```

Best Threshold

```
In [112... best_idx_rf = np.argmax(f1_scores_rf)  
  
best_threshold_rf = thresholds_rf[best_idx_rf]  
  
print("\nBest Threshold:", best_threshold_rf)
```

Best Threshold: 0.775

Final Predictions

```
In [113... y_pred_rf = (  
    y_prob_rf >= best_threshold_rf  
) .astype(int)
```

Evaluation

```
In [114... print("\nCONFUSION MATRIX\n")  
  
cm_rf = confusion_matrix(  
    y_test,  
    y_pred_rf  
)  
  
print(cm_rf)  
  
plt.figure(figsize=(6,4))
```

```
sns.heatmap(
    cm_rf,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.title("Random Forest Confusion Matrix")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()

print("\nCLASSIFICATION REPORT\n")

print(classification_report(
    y_test,
    y_pred_rf
))

print(
    "Precision:",
    precision_score(y_test, y_pred_rf)
)

print(
    "Recall:",
    recall_score(y_test, y_pred_rf)
)

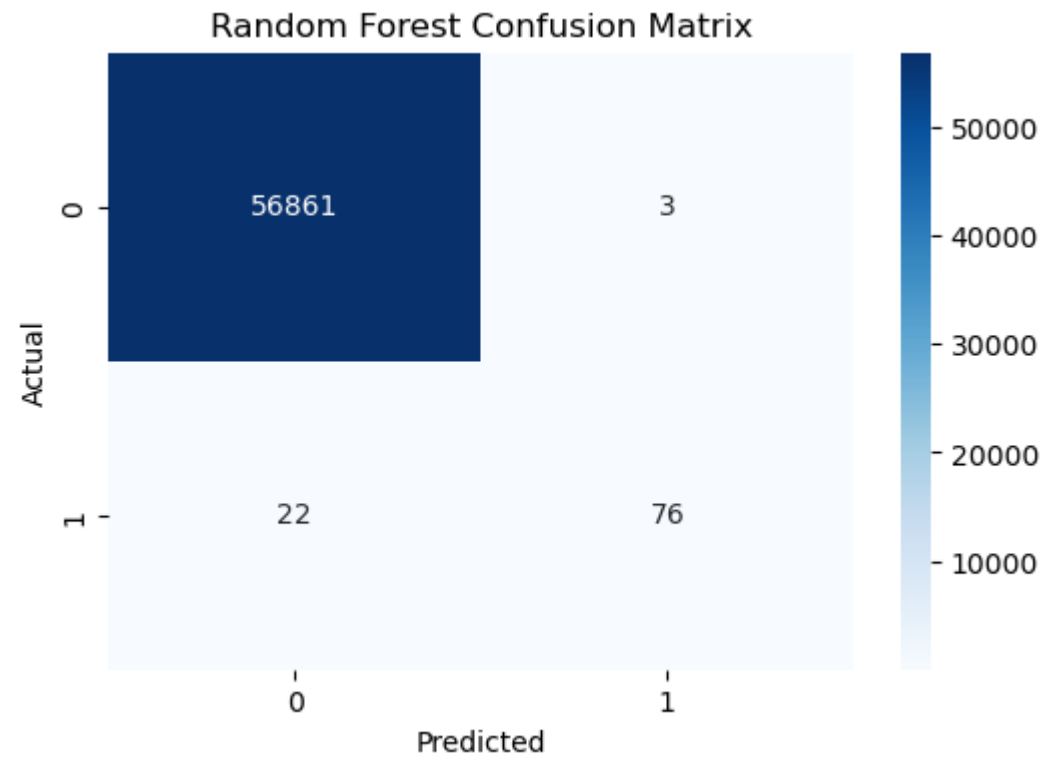
print(
    "F1 Score:",
    f1_score(y_test, y_pred_rf)
)

print(
    "ROC-AUC:",
```

```
roc_auc_score(y_test, y_prob_rf)  
)
```

CONFUSION MATRIX

```
[[56861   3]  
 [   22   76]]
```



CLASSIFICATION REPORT

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.96	0.78	0.86	98
accuracy			1.00	56962
macro avg	0.98	0.89	0.93	56962
weighted avg	1.00	1.00	1.00	56962

Precision: 0.9620253164556962

Recall: 0.7755102040816326

F1 Score: 0.8587570621468926

ROC-AUC: 0.9753984264640014

Feature Importance

```
In [115... importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": rf_model.feature_importances_
})

importance = importance.sort_values(
    by="Importance",
    ascending=False
)

print("\nTOP FEATURES\n")

print(importance.head(10))

plt.figure(figsize=(10,6))

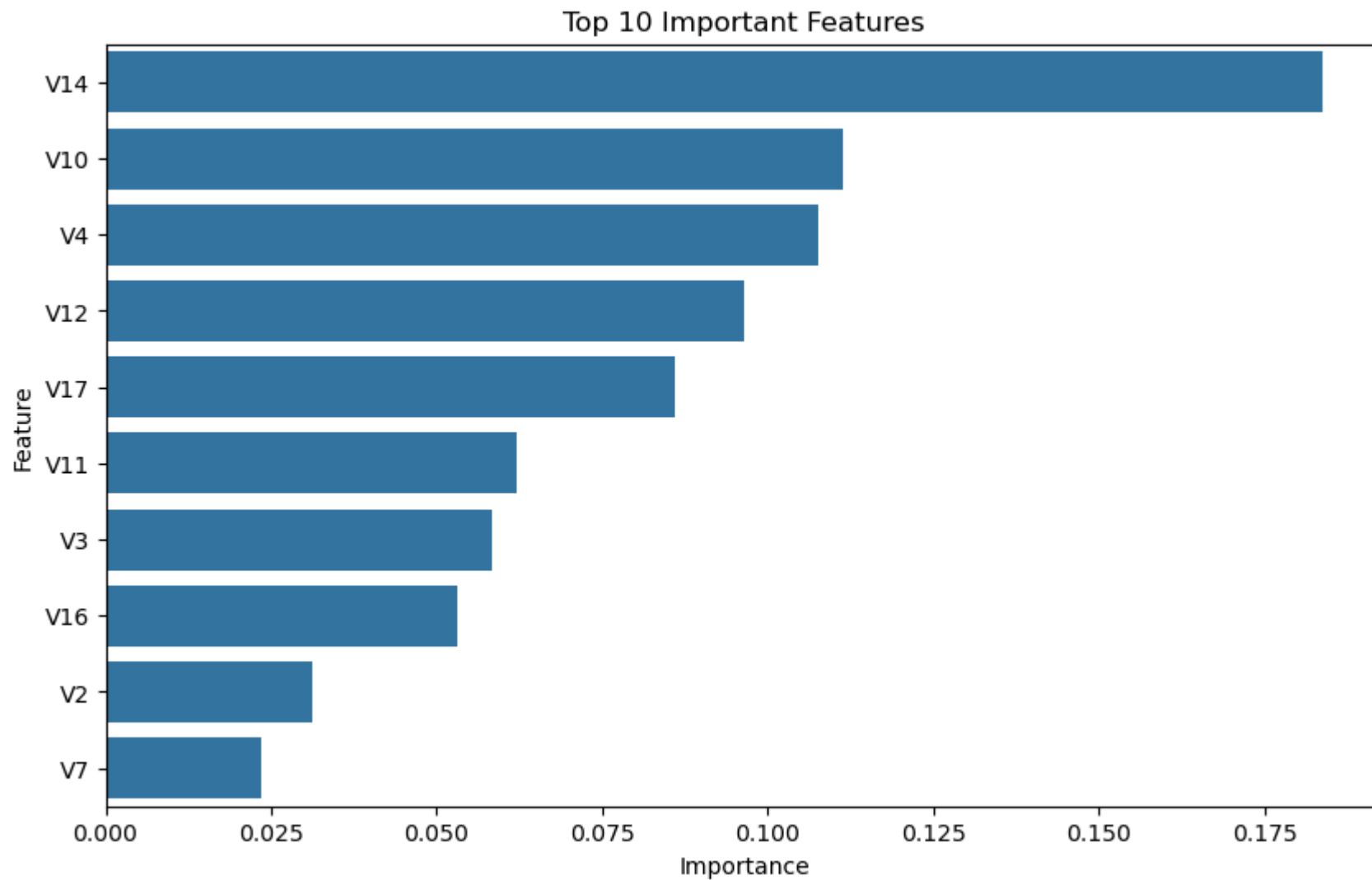
sns.barplot(
    x='Importance',
    y='Feature',
    data=importance.head(10)
)
```



```
plt.title("Top 10 Important Features")  
  
plt.show()
```

TOP FEATURES

	Feature	Importance
14	V14	0.183722
10	V10	0.111426
4	V4	0.107697
12	V12	0.096527
17	V17	0.086023
11	V11	0.062094
3	V3	0.058454
16	V16	0.053111
2	V2	0.031257
7	V7	0.023408



Save the Model

```
In [116... joblib.dump(rf_model, 'fraud_detection_rf.pkl')
```

```
Out[116... ['fraud_detection_rf.pkl']
```

XGBOOST MODEL

In [117...

```

print("\n" + "="*60)
print("XGBOOST")
print("="*60)

# -----
# Class Imbalance Weight
# -----

scale_pos_weight = (
    len(y_train[y_train == 0]) /
    len(y_train[y_train == 1])
)

# -----
# Train Model
# -----

xgb_model = XGBClassifier(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=5,
    subsample=0.8,
    colsample_bytree=0.8,
    scale_pos_weight=scale_pos_weight,
    random_state=42,
    eval_metric='logloss',
    n_jobs=-1
)

xgb_model.fit(
    X_train,
    y_train
)

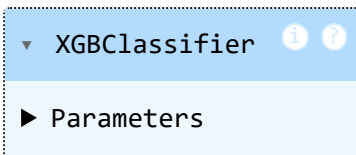
```

```

=====
XGBOOST
=====

```

Out[117...]



Predict Probabilities

```
In [118... y_prob_xgb = xgb_model.predict_proba(X_test)[:, 1]
```

Precision Recall Curve

```
In [119... precision_xgb, recall_xgb, thresholds_xgb = precision_recall_curve(  
    y_test,  
    y_prob_xgb  
)
```

F1 Scores

```
In [120... f1_scores_xgb = (  
    2 * precision_xgb[:-1] * recall_xgb[:-1]  
) / (  
    precision_xgb[:-1] + recall_xgb[:-1] + 1e-10  
)
```

Best Threshold

```
In [121... best_idx_xgb = np.argmax(f1_scores_xgb)  
  
best_threshold_xgb = thresholds_xgb[best_idx_xgb]  
  
print("\nBest Threshold:", best_threshold_xgb)
```

Best Threshold: 0.93744224

Final Predictions

```
In [122... y_pred_xgb = (  
    y_prob_xgb >= best_threshold_xgb  
) .astype(int)
```

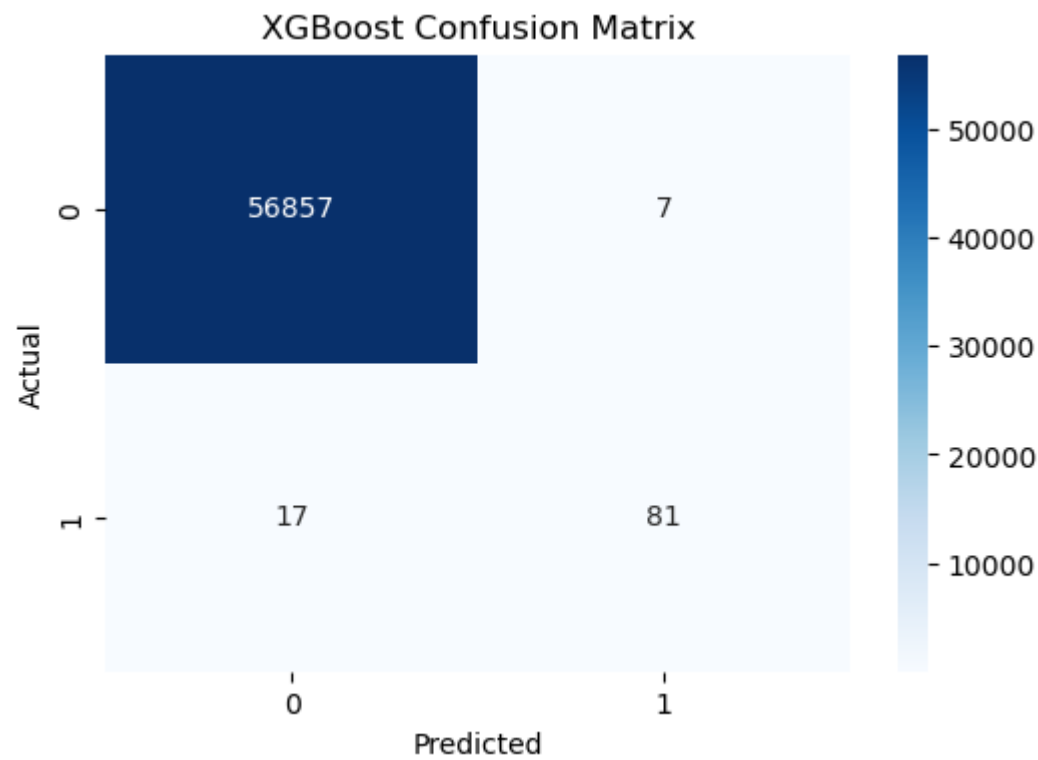
Evaluation

```
In [123... print("\nCONFUSION MATRIX\n")  
  
cm_xgb = confusion_matrix(  
    y_test,  
    y_pred_xgb  
)  
  
print(cm_xgb)  
  
plt.figure(figsize=(6,4))  
  
sns.heatmap(  
    cm_xgb,  
    annot=True,  
    fmt='d',  
    cmap='Blues'  
)  
  
plt.title("XGBoost Confusion Matrix")  
  
plt.xlabel("Predicted")  
  
plt.ylabel("Actual")  
  
plt.show()  
  
print("\nCLASSIFICATION REPORT\n")  
  
print(classification_report(  
    y_test,  
    y_pred_xgb  
)
```

```
print(  
    "Precision:",  
    precision_score(y_test, y_pred_xgb)  
)  
  
print(  
    "Recall:",  
    recall_score(y_test, y_pred_xgb)  
)  
  
print(  
    "F1 Score:",  
    f1_score(y_test, y_pred_xgb)  
)  
  
print(  
    "ROC-AUC:",  
    roc_auc_score(y_test, y_prob_xgb)  
)
```

CONFUSION MATRIX

```
[[56857    7]  
 [   17   81]]
```



CLASSIFICATION REPORT

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.92	0.83	0.87	98
accuracy			1.00	56962
macro avg	0.96	0.91	0.94	56962
weighted avg	1.00	1.00	1.00	56962

Precision: 0.9204545454545454
Recall: 0.826530612244898
F1 Score: 0.8709677419354839
ROC-AUC: 0.9829015596109011

Feed Forward Neural Network

Build Feed Forward Neural Network

Architecture: Input Layer/ Hidden Layer 1/ Hidden Layer 2/ Output Layer

In [124...

```
print("\n" + "="*60)
print("FEED FORWARD NEURAL NETWORK")
print("="*60)
```

```
# -----
# Class Weights
# -----
```

```
neg = len(y_train[y_train == 0])
pos = len(y_train[y_train == 1])
```

```
weight_for_0 = 1.0
weight_for_1 = neg / pos
```

```
class_weight = {
    0: weight_for_0,
    1: weight_for_1
}
```

```
# -----
# Build Model
# -----
```

```
ffnn_model = Sequential()
```

```
ffnn_model.add(
    Dense(
        64,
        activation='relu',
        input_dim=X_train.shape[1]
    )
)
```

```
ffnn_model.add(Dropout(0.3))
```



```
ffnn_model.add(Dense(32, activation='relu'))

ffnn_model.add(Dropout(0.3))

ffnn_model.add(Dense(16, activation='relu'))

ffnn_model.add(Dense(1, activation='sigmoid'))
```

```
=====
FEED FORWARD NEURAL NETWORK
=====
```

Compile the Model

```
In [125... ffnn_model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=[
        tf.keras.metrics.Precision(name='precision'),
        tf.keras.metrics.Recall(name='recall'),
        tf.keras.metrics.AUC(name='auc')
    ]
)
```

Early Stopping

```
In [126... early_stop = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)
```

Train the Neural Network

```
In [127... history = ffnn_model.fit(
    X_train,
    y_train,
```

```
validation_split=0.2,  
epochs=30,  
batch_size=256,  
class_weight=class_weight,  
callbacks=[early_stop],  
verbose=1  
)
```

Epoch 1/30

713/713  4s 3ms/step - auc: 0.9100 - loss: 0.8154 - precision: 0.0374 - recall: 0.7653 - val_auc: 0.9881 - val_loss: 0.1278 - val_precision: 0.0439 - val_recall: 0.9157

Epoch 2/30

713/713  2s 3ms/step - auc: 0.9704 - loss: 0.4103 - precision: 0.0334 - recall: 0.9068 - val_auc: 0.9912 - val_loss: 0.1109 - val_precision: 0.0552 - val_recall: 0.9036

Epoch 3/30

713/713  2s 3ms/step - auc: 0.9784 - loss: 0.3430 - precision: 0.0397 - recall: 0.9196 - val_auc: 0.9926 - val_loss: 0.1108 - val_precision: 0.0571 - val_recall: 0.9157

Epoch 4/30

713/713  2s 3ms/step - auc: 0.9867 - loss: 0.2846 - precision: 0.0415 - recall: 0.9035 - val_auc: 0.9922 - val_loss: 0.0752 - val_precision: 0.0795 - val_recall: 0.9036

Epoch 5/30

713/713  2s 3ms/step - auc: 0.9851 - loss: 0.2912 - precision: 0.0444 - recall: 0.9196 - val_auc: 0.9911 - val_loss: 0.1057 - val_precision: 0.0566 - val_recall: 0.9398

Epoch 6/30

713/713  2s 3ms/step - auc: 0.9908 - loss: 0.2303 - precision: 0.0509 - recall: 0.9453 - val_auc: 0.9869 - val_loss: 0.0747 - val_precision: 0.0781 - val_recall: 0.9157

Epoch 7/30

713/713  2s 3ms/step - auc: 0.9896 - loss: 0.2498 - precision: 0.0439 - recall: 0.9357 - val_auc: 0.9908 - val_loss: 0.0752 - val_precision: 0.0676 - val_recall: 0.9277

Epoch 8/30

713/713  2s 3ms/step - auc: 0.9919 - loss: 0.2181 - precision: 0.0471 - recall: 0.9357 - val_auc: 0.9921 - val_loss: 0.0651 - val_precision: 0.0718 - val_recall: 0.9398

Epoch 9/30

713/713  2s 3ms/step - auc: 0.9914 - loss: 0.2200 - precision: 0.0541 - recall: 0.9453 - val_auc: 0.9896 - val_loss: 0.0574 - val_precision: 0.0982 - val_recall: 0.9036

Epoch 10/30

713/713  2s 3ms/step - auc: 0.9928 - loss: 0.2149 - precision: 0.0547 - recall: 0.9421 - val_auc: 0.9920 - val_loss: 0.1407 - val_precision: 0.0353 - val_recall: 0.9639

Epoch 11/30

713/713  2s 3ms/step - auc: 0.9950 - loss: 0.1861 - precision: 0.0499 - recall: 0.9550 - val_auc: 0.9861 - val_loss: 0.0464 - val_precision: 0.0963 - val_recall: 0.9157

Epoch 12/30

713/713  2s 3ms/step - auc: 0.9949 - loss: 0.1825 - precision: 0.0608 - recall: 0.9550 - val_auc: 0.9874 - val_loss: 0.0599 - val_precision: 0.0852 - val_recall: 0.9398

Epoch 13/30

713/713  2s 3ms/step - auc: 0.9962 - loss: 0.1574 - precision: 0.0650 - recall: 0.9550 - val_auc: 0.9842 - val_loss: 0.0556 - val_precision: 0.0863 - val_recall: 0.9277

Epoch 14/30

713/713  2s 3ms/step - auc: 0.9916 - loss: 0.2255 - precision: 0.0549 - recall: 0.9582 - val_auc: 0.9834 -

val_loss: 0.0644 - val_precision: 0.0851 - val_recall: 0.9398
Epoch 15/30
713/713  2s 3ms/step - auc: 0.9943 - loss: 0.1794 - precision: 0.0585 - recall: 0.9678 - val_auc: 0.9875 -
val_loss: 0.0455 - val_precision: 0.1142 - val_recall: 0.9036
Epoch 16/30
713/713  2s 3ms/step - auc: 0.9966 - loss: 0.1482 - precision: 0.0634 - recall: 0.9678 - val_auc: 0.9855 -
val_loss: 0.0360 - val_precision: 0.1234 - val_recall: 0.9036
Epoch 17/30
713/713  2s 3ms/step - auc: 0.9977 - loss: 0.1236 - precision: 0.0828 - recall: 0.9711 - val_auc: 0.9806 -
val_loss: 0.0367 - val_precision: 0.1197 - val_recall: 0.8916
Epoch 18/30
713/713  2s 3ms/step - auc: 0.9975 - loss: 0.1299 - precision: 0.0707 - recall: 0.9711 - val_auc: 0.9858 -
val_loss: 0.0482 - val_precision: 0.0946 - val_recall: 0.9157
Epoch 19/30
713/713  2s 3ms/step - auc: 0.9972 - loss: 0.1347 - precision: 0.0695 - recall: 0.9711 - val_auc: 0.9846 -
val_loss: 0.0533 - val_precision: 0.0819 - val_recall: 0.9398
Epoch 20/30
713/713  2s 3ms/step - auc: 0.9979 - loss: 0.1165 - precision: 0.0692 - recall: 0.9743 - val_auc: 0.9815 -
val_loss: 0.0358 - val_precision: 0.1077 - val_recall: 0.9277
Epoch 21/30
713/713  2s 3ms/step - auc: 0.9957 - loss: 0.1725 - precision: 0.0701 - recall: 0.9743 - val_auc: 0.9848 -
val_loss: 0.0426 - val_precision: 0.1015 - val_recall: 0.9036
Epoch 22/30
713/713  3s 4ms/step - auc: 0.9981 - loss: 0.1119 - precision: 0.0751 - recall: 0.9775 - val_auc: 0.9711 -
val_loss: 0.0271 - val_precision: 0.1390 - val_recall: 0.8795
Epoch 23/30
713/713  2s 3ms/step - auc: 0.9974 - loss: 0.1315 - precision: 0.0737 - recall: 0.9646 - val_auc: 0.9807 -
val_loss: 0.0428 - val_precision: 0.0948 - val_recall: 0.9157
Epoch 24/30
713/713  2s 3ms/step - auc: 0.9982 - loss: 0.1024 - precision: 0.0680 - recall: 0.9871 - val_auc: 0.9818 -
val_loss: 0.0431 - val_precision: 0.0781 - val_recall: 0.8916
Epoch 25/30
713/713  3s 4ms/step - auc: 0.9964 - loss: 0.1406 - precision: 0.0687 - recall: 0.9743 - val_auc: 0.9810 -
val_loss: 0.0329 - val_precision: 0.1142 - val_recall: 0.9036
Epoch 26/30
713/713  2s 3ms/step - auc: 0.9962 - loss: 0.1341 - precision: 0.0800 - recall: 0.9678 - val_auc: 0.9795 -
val_loss: 0.0559 - val_precision: 0.0687 - val_recall: 0.9398
Epoch 27/30
713/713  3s 3ms/step - auc: 0.9983 - loss: 0.1040 - precision: 0.0660 - recall: 0.9807 - val_auc: 0.9759 -
val_loss: 0.0418 - val_precision: 0.0905 - val_recall: 0.9398

Predict Probabilities

```
In [128... y_prob_nn = ffnn_model.predict(X_test).ravel()
```

1781/1781  2s 1ms/step

Precision Recall Curve

```
In [129... precision_nn, recall_nn, thresholds_nn = precision_recall_curve(  
    y_test,  
    y_prob_nn  
)
```

F1 Scores

```
In [130... f1_scores_nn = (  
    2 * precision_nn[:-1] * recall_nn[:-1]  
) / (  
    precision_nn[:-1] + recall_nn[:-1] + 1e-10  
)
```

Best Threshold

```
In [131... best_idx_nn = np.argmax(f1_scores_nn)  
  
best_threshold_nn = thresholds_nn[best_idx_nn]  
  
print("\nBest Threshold:", best_threshold_nn)
```

Best Threshold: 0.99306524

Final Predictions

```
In [132... y_pred_nn = (  
    y_prob_nn >= best_threshold_nn
```

```
).astype(int)
```

Evaluate FFNN Model

```
In [133... print("\nCONFUSION MATRIX\n")

cm_nn = confusion_matrix(
    y_test,
    y_pred_nn
)

print(cm_nn)

plt.figure(figsize=(6,4))

sns.heatmap(
    cm_nn,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.title("FFNN Confusion Matrix")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()

print("\nCLASSIFICATION REPORT\n")

print(classification_report(
    y_test,
    y_pred_nn
))

print(
    "Precision:",
```

```
precision_score(y_test, y_pred_nn)
)

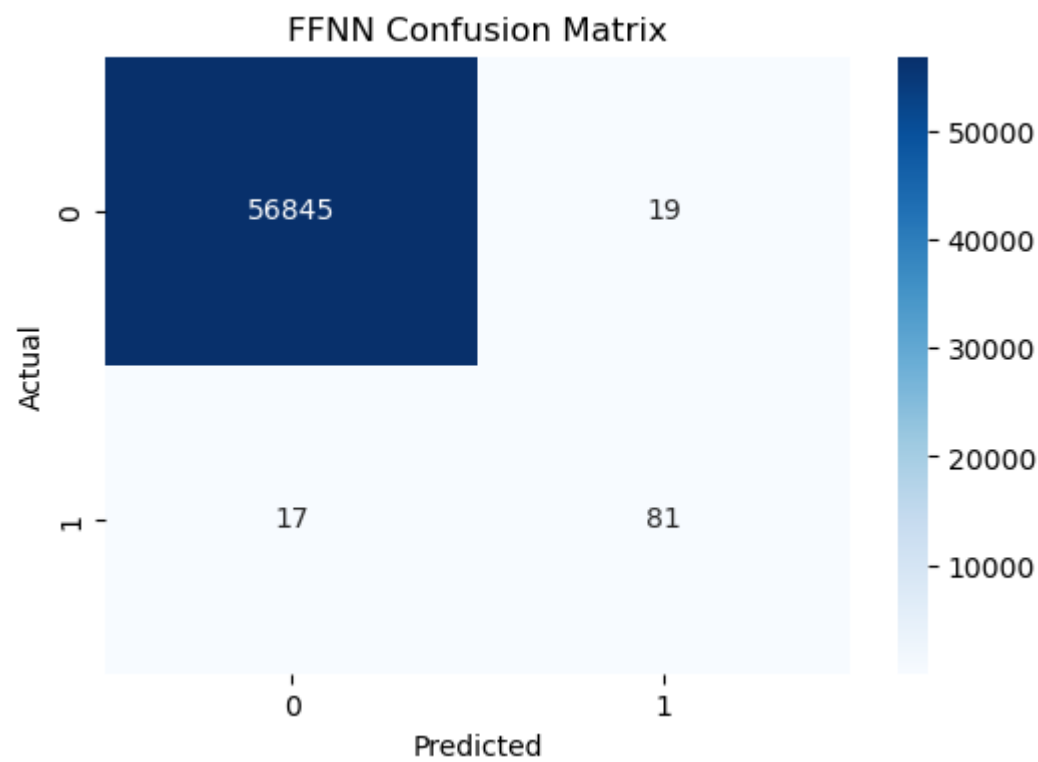
print(
    "Recall:",
    recall_score(y_test, y_pred_nn)
)

print(
    "F1 Score:",
    f1_score(y_test, y_pred_nn)
)

print(
    "ROC-AUC:",
    roc_auc_score(y_test, y_prob_nn)
)
```

CONFUSION MATRIX

```
[[56845  19]
 [  17  81]]
```



CLASSIFICATION REPORT

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.81	0.83	0.82	98
accuracy			1.00	56962
macro avg	0.90	0.91	0.91	56962
weighted avg	1.00	1.00	1.00	56962

Precision: 0.81
Recall: 0.826530612244898
F1 Score: 0.8181818181818182
ROC-AUC: 0.9821542520356482

TRAINING CURVES


```
In [134... plt.figure(figsize=(8,5))

plt.plot(history.history['loss'])

plt.plot(history.history['val_loss'])

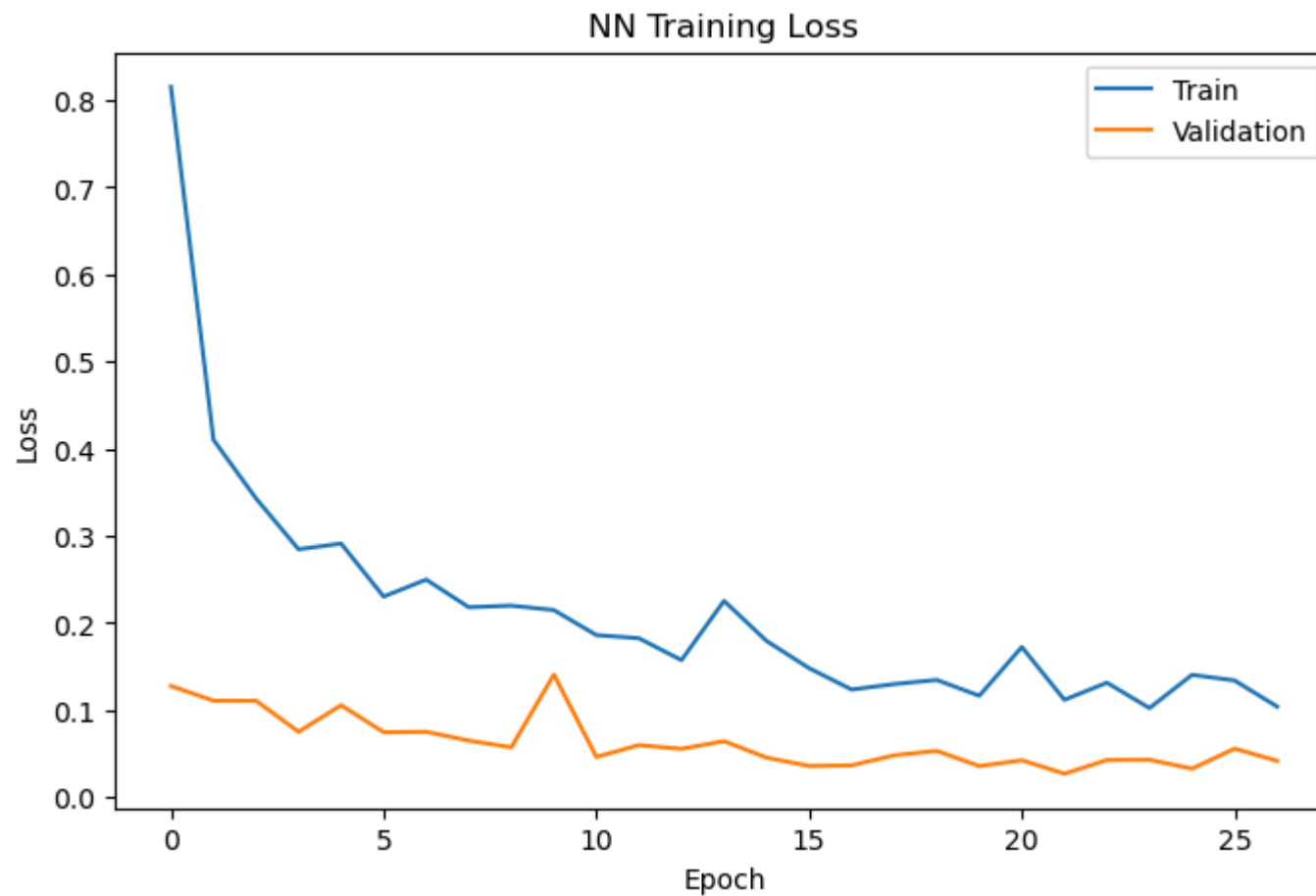
plt.title("NN Training Loss")

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.legend(['Train', 'Validation'])

plt.show()
```



Save Neural Network Model

```
In [135... ffnn_model.save("fraud_detection_ffnn.h5")
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

Autoencoder

Input layer

In [136...

```

print("\n" + "="*60)
print("AUTOENCODER")
print("="*60)

# -----
# Train ONLY on Normal Transactions
# -----

X_train_normal = X_train[
    y_train == 0
]

input_dim = X_train.shape[1]

input_layer = Input(shape=(input_dim,))

=====
AUTOENCODER
=====

```

Encoder

In [137...

```

encoder = Dense(
    16,
    activation='relu'
)(input_layer)

encoder = Dense(
    8,
    activation='relu'
)(encoder)

```

Decoder

In [138...

```

decoder = Dense(
    16,

```

```
        activation='relu'  
    )(encoder)  
  
    decoder = Dense(  
        input_dim,  
        activation='linear'  
    )(decoder)
```

Build Model

```
In [139... autoencoder = Model(  
    inputs=input_layer,  
    outputs=decoder  
)
```

Compile Model

```
In [140... autoencoder.compile(  
    optimizer='adam',  
    loss='mse'  
)
```

Train Model

```
In [141... history_auto = autoencoder.fit(  
    X_train_normal,  
    X_train_normal,  
    epochs=20,  
    batch_size=256,  
    validation_split=0.2,  
    verbose=1  
)
```

Epoch 1/20
711/711  3s 3ms/step - loss: 0.7794 - val_loss: 0.5418
Epoch 2/20
711/711  2s 3ms/step - loss: 0.4577 - val_loss: 0.3890
Epoch 3/20
711/711  2s 3ms/step - loss: 0.3689 - val_loss: 0.3474
Epoch 4/20
711/711  2s 3ms/step - loss: 0.3393 - val_loss: 0.3292
Epoch 5/20
711/711  2s 3ms/step - loss: 0.3243 - val_loss: 0.3170
Epoch 6/20
711/711  2s 3ms/step - loss: 0.3133 - val_loss: 0.3085
Epoch 7/20
711/711  2s 3ms/step - loss: 0.3046 - val_loss: 0.3009
Epoch 8/20
711/711  2s 2ms/step - loss: 0.2983 - val_loss: 0.2948
Epoch 9/20
711/711  2s 2ms/step - loss: 0.2928 - val_loss: 0.2904
Epoch 10/20
711/711  2s 2ms/step - loss: 0.2881 - val_loss: 0.2854
Epoch 11/20
711/711  2s 2ms/step - loss: 0.2832 - val_loss: 0.2813
Epoch 12/20
711/711  2s 2ms/step - loss: 0.2795 - val_loss: 0.2773
Epoch 13/20
711/711  2s 2ms/step - loss: 0.2766 - val_loss: 0.2746
Epoch 14/20
711/711  2s 2ms/step - loss: 0.2734 - val_loss: 0.2719
Epoch 15/20
711/711  2s 2ms/step - loss: 0.2711 - val_loss: 0.2693
Epoch 16/20
711/711  2s 2ms/step - loss: 0.2675 - val_loss: 0.2651
Epoch 17/20
711/711  2s 3ms/step - loss: 0.2652 - val_loss: 0.2660
Epoch 18/20
711/711  2s 2ms/step - loss: 0.2629 - val_loss: 0.2610
Epoch 19/20
711/711  2s 3ms/step - loss: 0.2623 - val_loss: 0.2617
Epoch 20/20
711/711  2s 3ms/step - loss: 0.2608 - val_loss: 0.2613

Reconstruction

```
In [142... reconstructed = autoencoder.predict(  
    X_test  
)
```

1781/1781  2s 1ms/step

Reconstruction Error

```
In [143... mse = np.mean(  
    np.power(  
        X_test - reconstructed,  
        2  
    ),  
    axis=1  
)
```

Threshold

```
In [144... threshold_auto = np.percentile(  
    mse,  
    95  
)  
  
print("\nThreshold:", threshold_auto)
```

Threshold: 0.5835904606822326

Final Predictions

```
In [145... y_pred_auto = (  
    mse > threshold_auto  
) .astype(int)
```

Evaluation

```
In [146... print("\nCONFUSION MATRIX\n")

cm_auto = confusion_matrix(
    y_test,
    y_pred_auto
)

print(cm_auto)

plt.figure(figsize=(6,4))

sns.heatmap(
    cm_auto,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.title("Autoencoder Confusion Matrix")

plt.xlabel("Predicted")

plt.ylabel("Actual")

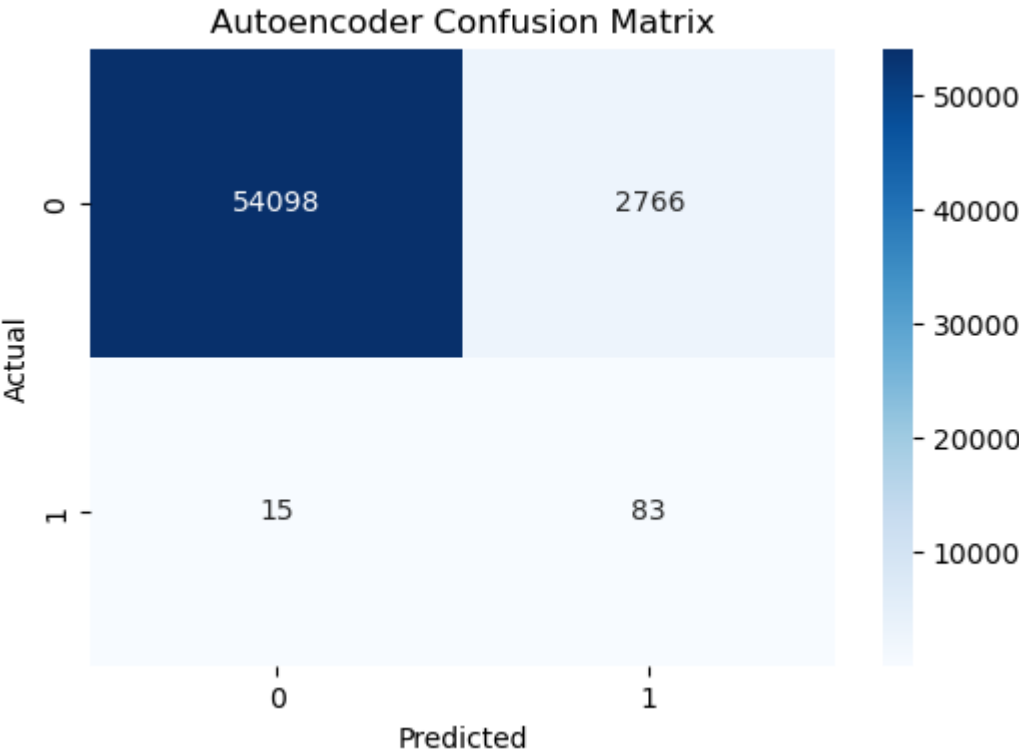
plt.show()

print("\nCLASSIFICATION REPORT\n")

print(classification_report(
    y_test,
    y_pred_auto
))
```

CONFUSION MATRIX

```
[[54098  2766]
 [   15    83]]
```



CLASSIFICATION REPORT

	precision	recall	f1-score	support
0	1.00	0.95	0.97	56864
1	0.03	0.85	0.06	98
accuracy			0.95	56962
macro avg	0.51	0.90	0.52	56962
weighted avg	1.00	0.95	0.97	56962

ENSEMBLE MODEL WITH DYNAMIC WEIGHTS

```
In [147... print("\n" + "="*60)
print("ENSEMBLE MODEL")
```



```
print("="*60)

# =====
# STEP 1: CALCULATE PR-AUC FOR EACH MODEL
# PR-AUC is better for highly imbalanced fraud datasets
# =====

# Logistic Regression PR-AUC
pr_auc_lr = auc(
    recall_lr,
    precision_lr
)

# Random Forest PR-AUC
pr_auc_rf = auc(
    recall_rf,
    precision_rf
)

# XGBoost PR-AUC
pr_auc_xgb = auc(
    recall_xgb,
    precision_xgb
)

# Neural Network PR-AUC
pr_auc_nn = auc(
    recall_nn,
    precision_nn
)

# =====
# STEP 2: TOTAL SCORE
# =====

total_pr_auc = (
    pr_auc_lr +
    pr_auc_rf +
    pr_auc_xgb +
    pr_auc_nn
)
```

```

# =====
# STEP 3: DYNAMIC WEIGHTS
# =====

w_lr = pr_auc_lr / total_pr_auc

w_rf = pr_auc_rf / total_pr_auc

w_xgb = pr_auc_xgb / total_pr_auc

w_nn = pr_auc_nn / total_pr_auc

# =====
# DISPLAY DYNAMIC WEIGHTS
# =====

print("\nDYNAMIC ENSEMBLE WEIGHTS\n")

print(f"Logistic Regression Weight : {w_lr:.4f}")

print(f"Random Forest Weight      : {w_rf:.4f}")

print(f"XGBoost Weight              : {w_xgb:.4f}")

print(f"Neural Network Weight       : {w_nn:.4f}")

```

```

=====
ENSEMBLE MODEL
=====

```

DYNAMIC ENSEMBLE WEIGHTS

```

Logistic Regression Weight : 0.2380
Random Forest Weight      : 0.2708
XGBoost Weight            : 0.2684
Neural Network Weight     : 0.2228

```

WEIGHTED ENSEMBLE PROBABILITIES

```
In [148... ensemble_prob = (  
    w_lr * y_prob_lr +  
    w_rf * y_prob_rf +  
    w_xgb * y_prob_xgb +  
    w_nn * y_prob_nn  
)
```

PRECISION-RECALL CURVE

```
In [149... precision_en, recall_en, thresholds_en = precision_recall_curve(  
    y_test,  
    ensemble_prob  
)
```

F1 SCORES FOR ALL THRESHOLDS

```
In [150... f1_scores_en = (  
    2 * precision_en[:-1] * recall_en[:-1]  
) / (  
    precision_en[:-1] + recall_en[:-1] + 1e-10  
)
```

BEST THRESHOLD

```
In [151... best_idx_en = np.argmax(  
    f1_scores_en  
)  
  
best_threshold_en = thresholds_en[  
    best_idx_en  
]  
  
print("\nBEST ENSEMBLE THRESHOLD:",  
    best_threshold_en)
```

BEST ENSEMBLE THRESHOLD: 0.8436913321089801

FINAL PREDICTIONS

```
In [152... y_pred_ensemble = (  
    ensemble_prob >= best_threshold_en  
)<div data-bbox="129 231 281 257" data-label="Section-Header">

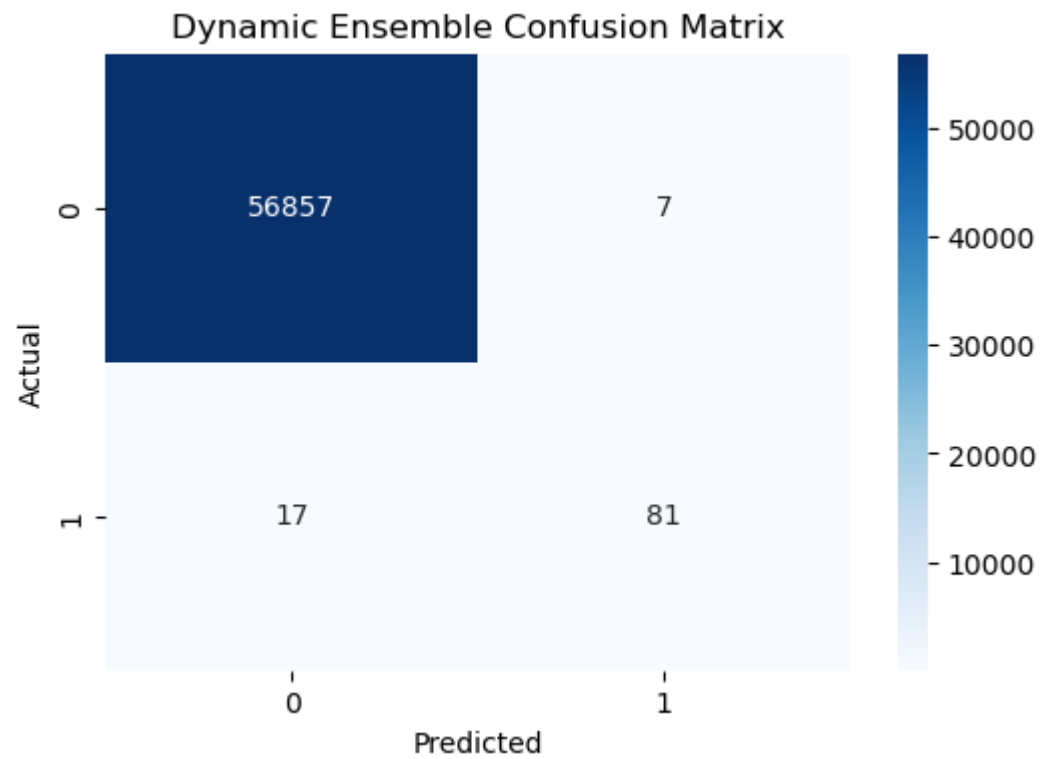
## CONFUSION MATRIX


```

```
In [153... print("\nCONFUSION MATRIX\n")  
  
cm_en = confusion_matrix(  
    y_test,  
    y_pred_ensemble  
)  
  
print(cm_en)  
  
plt.figure(figsize=(6,4))  
  
sns.heatmap(  
    cm_en,  
    annot=True,  
    fmt='d',  
    cmap='Blues'  
)  
  
plt.title("Dynamic Ensemble Confusion Matrix")  
  
plt.xlabel("Predicted")  
  
plt.ylabel("Actual")  
  
plt.show()
```

CONFUSION MATRIX

```
[[56857    7]  
 [   17   81]]
```



CLASSIFICATION REPORT

```
In [154... print("\nCLASSIFICATION REPORT\n")

print(classification_report(
    y_test,
    y_pred_ensemble
))

print(
    "Precision:",
    precision_score(
        y_test,
        y_pred_ensemble
    )
)
```

```

)

print(
    "Recall:",
    recall_score(
        y_test,
        y_pred_ensemble
    )
)

print(
    "F1 Score:",
    f1_score(
        y_test,
        y_pred_ensemble
    )
)

```

CLASSIFICATION REPORT

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.92	0.83	0.87	98
accuracy			1.00	56962
macro avg	0.96	0.91	0.94	56962
weighted avg	1.00	1.00	1.00	56962

Precision: 0.9204545454545454

Recall: 0.826530612244898

F1 Score: 0.8709677419354839

ROC-AUC

```

In [155... ensemble_roc_auc = roc_auc_score(
    y_test,
    ensemble_prob
)

```

```
print(  
    "ROC-AUC:",  
    ensemble_roc_auc  
)
```

ROC-AUC: 0.9743326002319894

ROC CURVE WITH BEST THRESHOLD

```
In [156... fpr_en, tpr_en, roc_thresholds_en = roc_curve(  
    y_test,  
    ensemble_prob  
)  
  
# Find Threshold Point on ROC Curve  
roc_best_idx_en = np.argmin(  
    np.abs(  
        roc_thresholds_en -  
        best_threshold_en  
    )  
)  
  
best_fpr_en = fpr_en[roc_best_idx_en]  
  
best_tpr_en = tpr_en[roc_best_idx_en]  
  
# Plot ROC Curve  
plt.figure(figsize=(8,6))  
  
plt.plot(  
    fpr_en,  
    tpr_en,  
    linewidth=2,  
    label=f"AUC = {ensemble_roc_auc:.4f}"  
)  
  
# Random Guess Line  
plt.plot(  
    [0,1],  
    [0,1],
```

```
        linestyle='--'
    )

    # Best Threshold Point
    plt.scatter(
        best_fpr_en,
        best_tpr_en,
        s=150,
        marker='o',
        label=f"Best Threshold = {best_threshold_en:.4f}"
    )

    # Annotate Threshold
    plt.annotate(
        f"Threshold = {best_threshold_en:.4f}",
        (best_fpr_en, best_tpr_en),
        textcoords="offset points",
        xytext=(15, -10),
        fontsize=10
    )

    plt.xlabel("False Positive Rate")

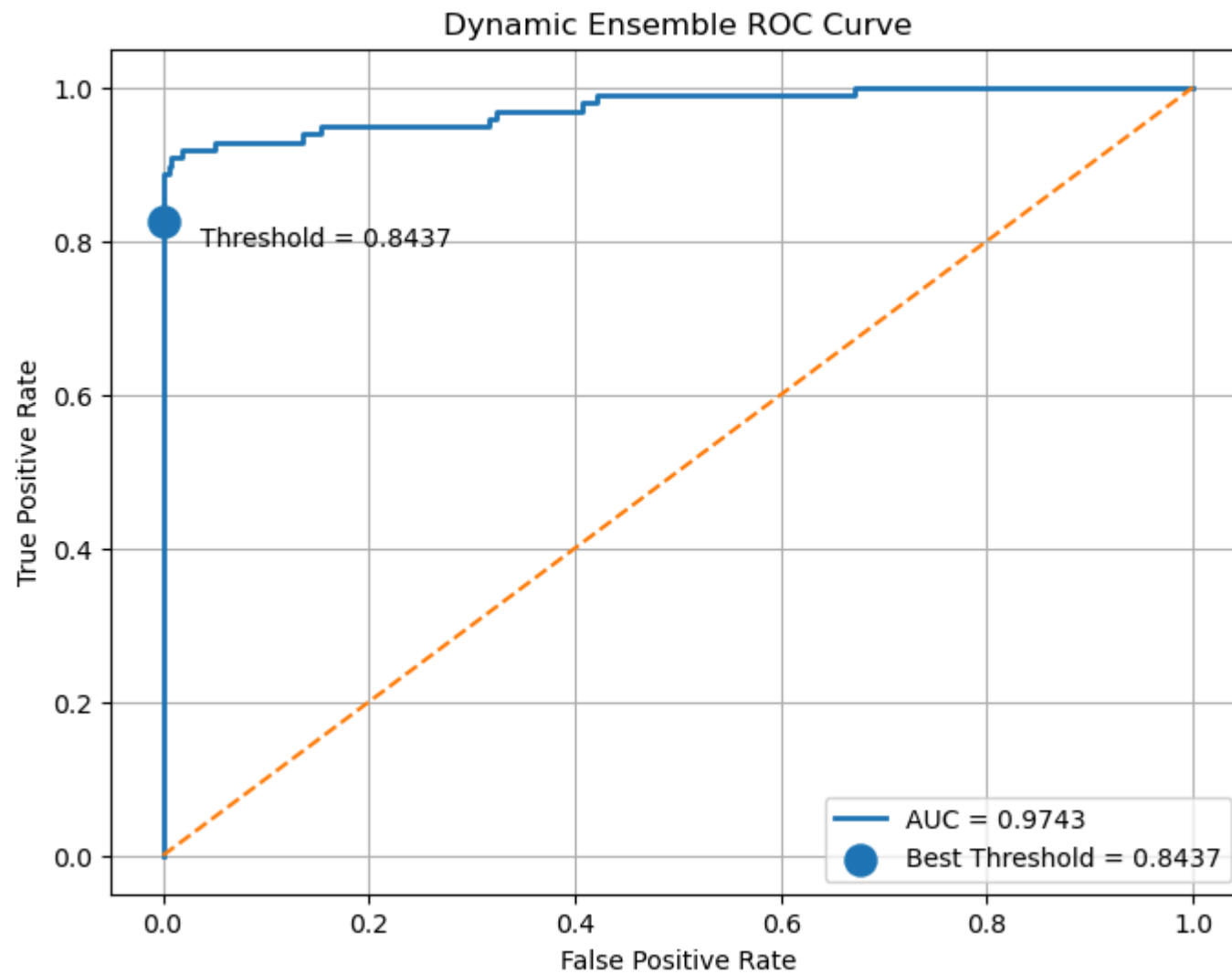
    plt.ylabel("True Positive Rate")

    plt.title("Dynamic Ensemble ROC Curve")

    plt.legend()

    plt.grid(True)

    plt.show()
```

PRECISION-RECALL CURVE

```
In [157... ensemble_pr_auc = auc(  
    recall_en,  
    precision_en  
)
```

```
plt.figure(figsize=(8,6))

plt.plot(
    recall_en,
    precision_en,
    linewidth=2,
    label=f"PR-AUC = {ensemble_pr_auc:.4f}"
)

# Best Threshold Point
plt.scatter(
    recall_en[best_idx_en],
    precision_en[best_idx_en],
    s=150,
    marker='o',
    label=f"Best Threshold = {best_threshold_en:.4f}"
)

# Annotate Threshold
plt.annotate(
    f"Threshold = {best_threshold_en:.4f}",
    (
        recall_en[best_idx_en],
        precision_en[best_idx_en]
    ),
    textcoords="offset points",
    xytext=(15, -10),
    fontsize=10
)

plt.xlabel("Recall")

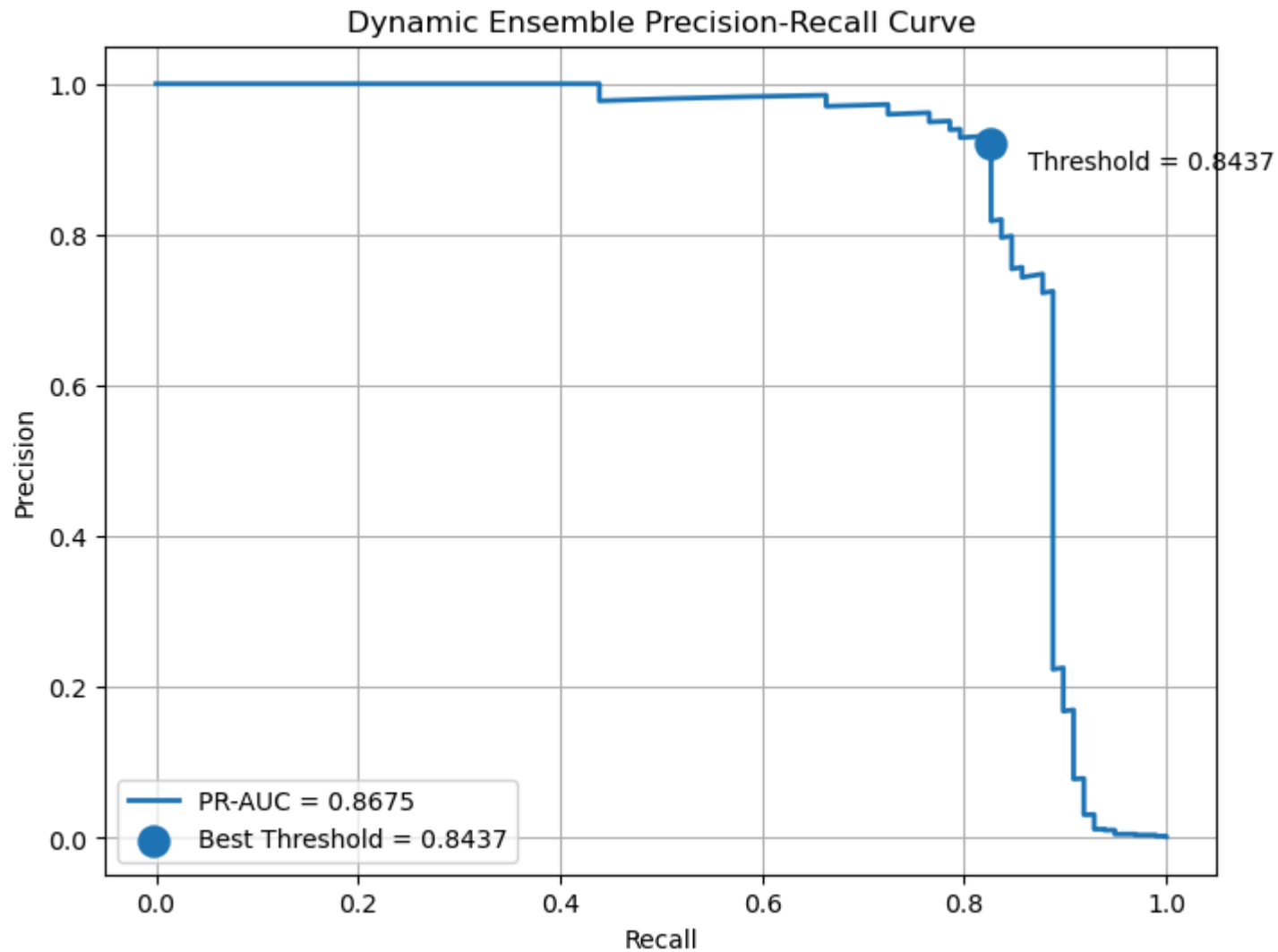
plt.ylabel("Precision")

plt.title("Dynamic Ensemble Precision-Recall Curve")

plt.legend()

plt.grid(True)
```

```
plt.show()
```



SAVE ENSEMBLE MODEL OUTPUTS

```
In [158... joblib.dump(  
    {
```

```

        "weights": {
            "lr": w_lr,
            "rf": w_rf,
            "xgb": w_xgb,
            "nn": w_nn
        },
        "threshold": best_threshold_en
    },
    "dynamic_ensemble_config.pkl"
)

print("\nDynamic Ensemble Model Completed Successfully!")

```

Dynamic Ensemble Model Completed Successfully!

STACKING ENSEMBLE WITH AUTOENCODER

In [159...

```

print("\n" + "="*60)
print("STACKING ENSEMBLE MODEL")
print("="*60)

# =====
# NORMALIZE AUTOENCODER MSE
# =====

from sklearn.preprocessing import MinMaxScaler

mse_scaler = MinMaxScaler()

mse_scaled = mse_scaler.fit_transform(
    mse.values.reshape(-1,1)
).ravel()

```

```

=====
STACKING ENSEMBLE MODEL
=====

```

CREATE META FEATURES

```
In [160... meta_features = pd.DataFrame({
    "lr_prob": y_prob_lr,
    "rf_prob": y_prob_rf,
    "xgb_prob": y_prob_xgb,
    "nn_prob": y_prob_nn,
    "autoencoder_score": mse_scaled
})

print("\nMETA FEATURE SAMPLE\n")

print(meta_features.head())
```

META FEATURE SAMPLE

	lr_prob	rf_prob	xgb_prob	nn_prob	autoencoder_score
0	0.002599	0.00	0.000027	5.762812e-06	0.000712
1	0.049992	0.00	0.000020	6.717738e-06	0.029293
2	0.000010	0.02	0.000210	8.378715e-07	0.006055
3	0.011821	0.00	0.000009	6.302818e-06	0.000605
4	0.980843	0.00	0.002094	5.710126e-01	0.001115

TRAIN META MODEL

```
In [161... X_meta_train, X_meta_test, y_meta_train, y_meta_test = train_test_split(
    meta_features,
    y_test,
    test_size=0.2,
    stratify=y_test,
    random_state=42
)
```

META MODEL

```
In [162... meta_model = LogisticRegression(  
    max_iter=1000,  
    random_state=42  
)  
  
meta_model.fit(  
    X_meta_train,  
    y_meta_train  
)
```

Out[162...

▼ LogisticRegression ⓘ ?
► Parameters

META MODEL PREDICTIONS

```
In [163... y_prob_stack = meta_model.predict_proba(  
    X_meta_test  
)[:,1]
```

PRECISION-RECALL CURVE

```
In [164... precision_stack, recall_stack, thresholds_stack = precision_recall_curve(  
  
    y_meta_test,  
    y_prob_stack  
)
```

F1 SCORES

```
In [165... f1_scores_stack = (  
    2 * precision_stack[:-1] * recall_stack[:-1]  
    ) / (  
    precision_stack[:-1] +  
    recall_stack[:-1] +  
    1e-10  
    )
```

BEST THRESHOLD

```
In [166... best_idx_stack = np.argmax(  
    f1_scores_stack  
    )  
  
    best_threshold_stack = thresholds_stack[  
        best_idx_stack  
    ]  
  
    print("\nBEST STACKING THRESHOLD:",  
        best_threshold_stack)
```

BEST STACKING THRESHOLD: 0.7495456829808383

FINAL PREDICTIONS

```
In [167... y_pred_stack = (  
    y_prob_stack >= best_threshold_stack  
    ).astype(int)
```

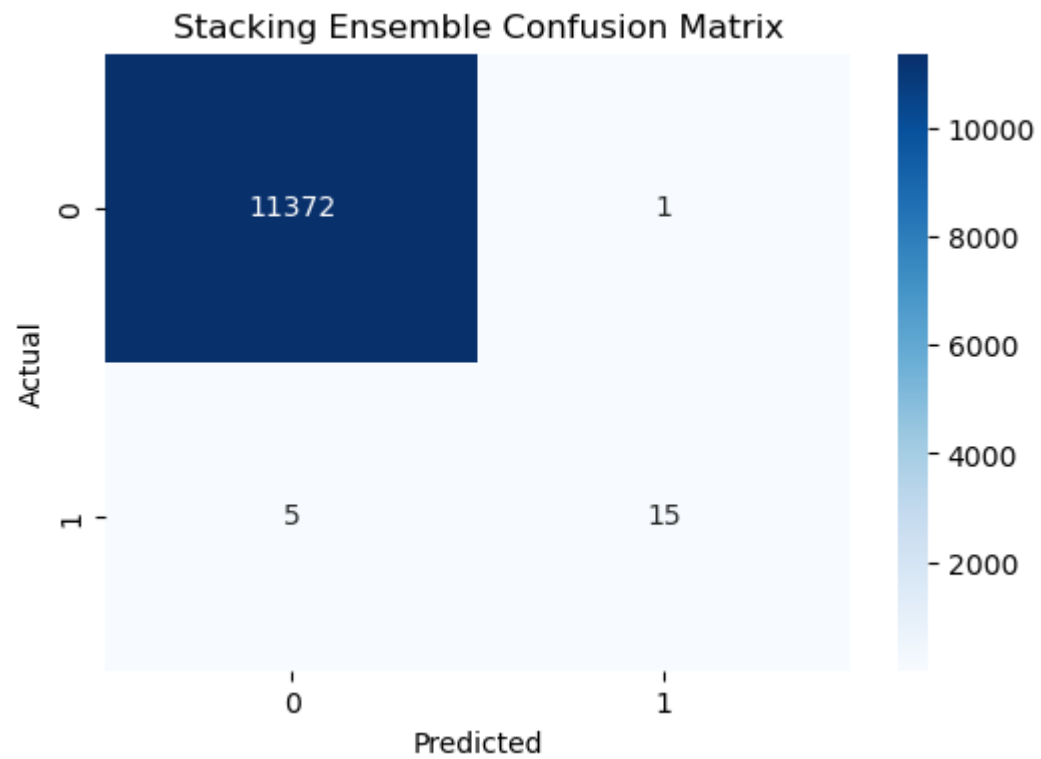
EVALUATION

```
In [168... print("\nCONFUSION MATRIX\n")
```

```
cm_stack = confusion_matrix(  
    y_meta_test,  
    y_pred_stack  
)  
  
print(cm_stack)  
  
plt.figure(figsize=(6,4))  
  
sns.heatmap(  
    cm_stack,  
    annot=True,  
    fmt='d',  
    cmap='Blues'  
)  
  
plt.title("Stacking Ensemble Confusion Matrix")  
  
plt.xlabel("Predicted")  
  
plt.ylabel("Actual")  
  
plt.show()
```

CONFUSION MATRIX

```
[[11372    1]  
 [     5   15]]
```

CLASSIFICATION REPORT

```
In [169... print(classification_report(  
    y_meta_test,  
    y_pred_stack  
)  
  
print(  
    "Precision:",  
    precision_score(  
        y_meta_test,  
        y_pred_stack  
    )  
)
```

```
print(
    "Recall:",
    recall_score(
        y_meta_test,
        y_pred_stack
    )
)

print(
    "F1 Score:",
    f1_score(
        y_meta_test,
        y_pred_stack
    )
)
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	11373
1	0.94	0.75	0.83	20
accuracy			1.00	11393
macro avg	0.97	0.87	0.92	11393
weighted avg	1.00	1.00	1.00	11393

Precision: 0.9375

Recall: 0.75

F1 Score: 0.8333333333333334

ROC-AUC

```
In [170... stack_roc_auc = roc_auc_score(
    y_meta_test,
    y_prob_stack
)

print(
    "ROC-AUC:",
    stack_roc_auc
)
```

ROC-AUC: 0.9989096984085114

ROC CURVE

```
In [171... # FIND ROC POINT CLOSEST TO BEST THRESHOLD
# =====

roc_best_idx_stack = np.argmin(
    np.abs(
        roc_thresholds_stack -
        best_threshold_stack
    )
)

best_fpr_stack = fpr_stack[
    roc_best_idx_stack
]

best_tpr_stack = tpr_stack[
    roc_best_idx_stack
]

# =====
# ROC CURVE
# =====

plt.figure(figsize=(8,6))

plt.plot(
    fpr_stack,
    tpr_stack,
    linewidth=2,
    label=f"AUC = {stack_roc_auc:.4f}"
)

plt.plot(
    [0,1],
    [0,1],
    linestyle='--'
)
```

```
# BEST THRESHOLD POINT
plt.scatter(

    best_fpr_stack,
    best_tpr_stack,

    s=150,

    label=f"Threshold={best_threshold_stack:.4f}"
)

# ANNOTATION
plt.annotate(

    f"Threshold={best_threshold_stack:.4f}",

    (
        best_fpr_stack,
        best_tpr_stack
    ),

    textcoords="offset points",

    xytext=(15, -10),

    fontsize=10
)

plt.xlabel("False Positive Rate")

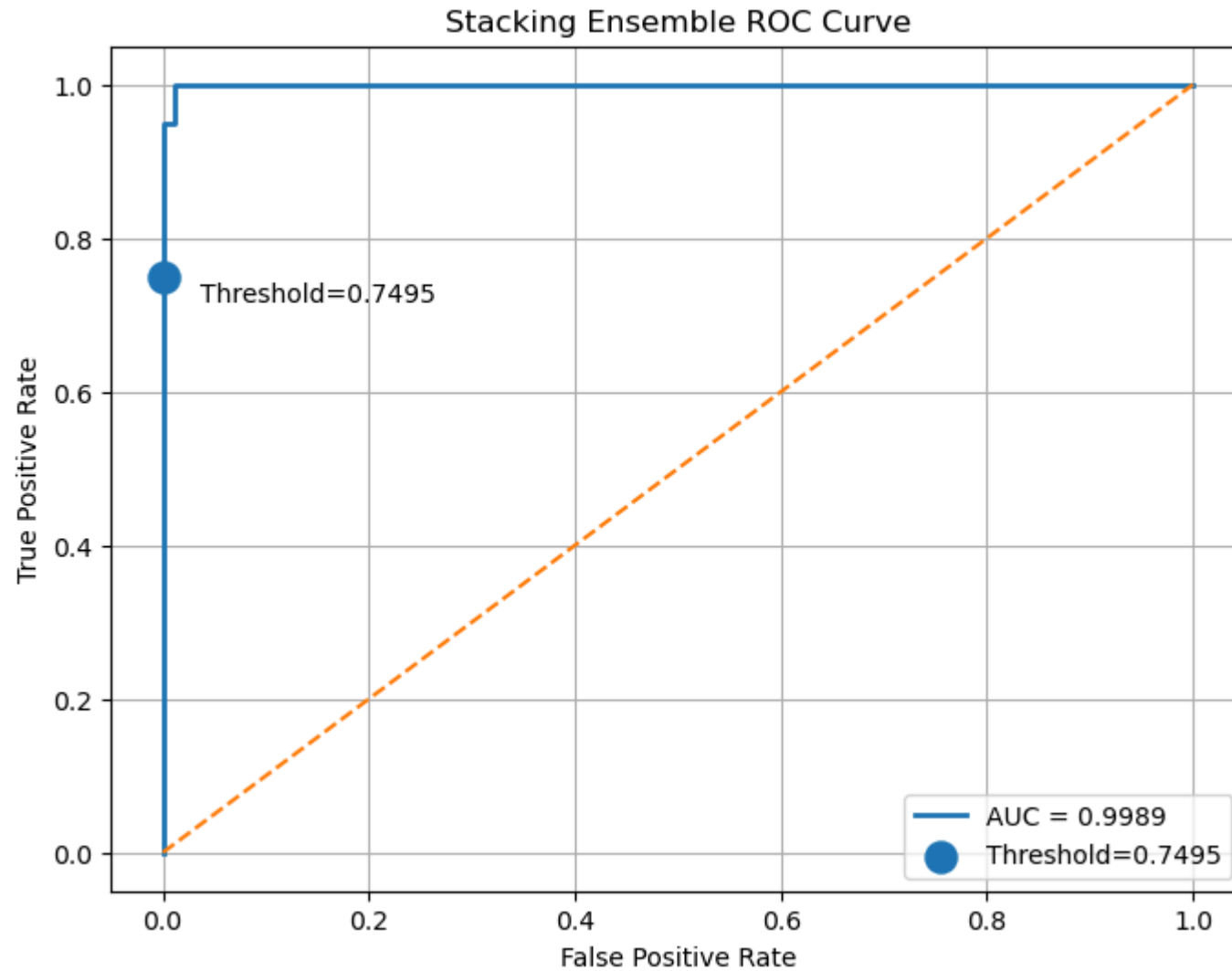
plt.ylabel("True Positive Rate")

plt.title("Stacking Ensemble ROC Curve")

plt.legend()

plt.grid(True)
```

```
plt.show()
```



PRECISION-RECALL CURVE

```
In [172... stack_pr_auc = auc(  
    recall_stack,
```

```
    precision_stack
)

plt.figure(figsize=(8,6))

plt.plot(

    recall_stack,
    precision_stack,

    linewidth=2,

    label=f"PR-AUC = {stack_pr_auc:.4f}"

)

plt.scatter(

    recall_stack[best_idx_stack],
    precision_stack[best_idx_stack],

    s=150,

    label=f"Threshold={best_threshold_stack:.4f}"

)

plt.xlabel("Recall")

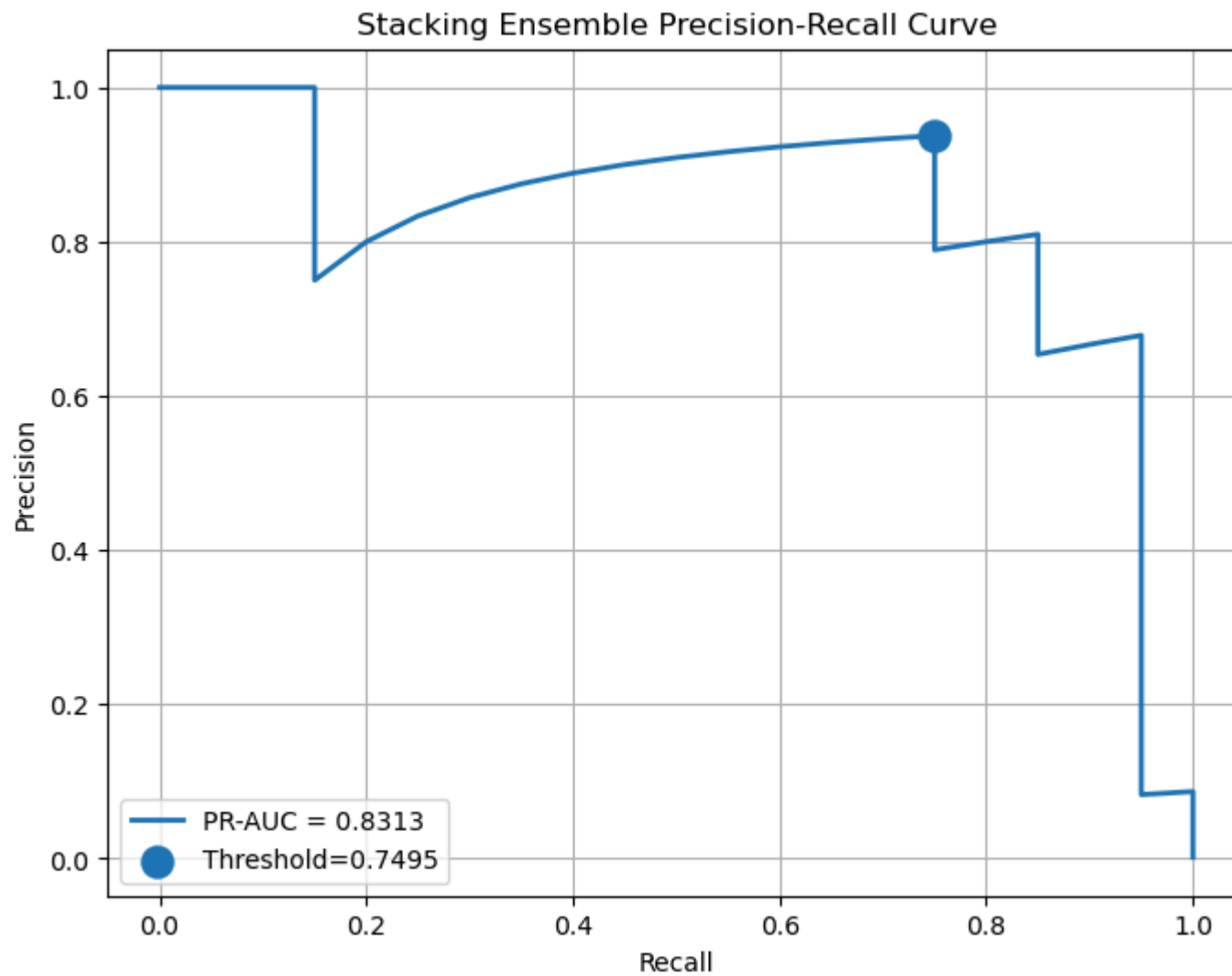
plt.ylabel("Precision")

plt.title("Stacking Ensemble Precision-Recall Curve")

plt.legend()

plt.grid(True)

plt.show()
```



SAVE STACKING MODEL

```
In [173... joblib.dump(  
    meta_model,  
    "stacking_meta_model.pkl"  
)
```

```
joblib.dump(  
    mse_scaler,  
    "autoencoder_mse_scaler.pkl"  
)  
  
print("\nStacking Ensemble Completed Successfully!")
```

Stacking Ensemble Completed Successfully!